



26. Python: Read and Calculate IP Address

"Networks are not just machines connected together... they are relationships between addresses."

Previous Section:

[!\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\) 25. Python: Multi-Dimensional Lists](#)

Contents:

[1. IPv4 Addresses](#)

[Creating an IPv4 Address in Python](#)

[Comparing IPv4 Addresses](#)

[Sorting IPv4 Addresses](#)

[2. Networks, CIDR, and Host Calculations](#)

[Understanding CIDR Notation](#)

[Working with Networks in Python](#)

[IP Address Classes](#)

[Public and Private IP Addresses](#)

[Subnet Masks and Host Masks](#)

[Calculating Hosts](#)

[Returning Individual Host Addresses](#)

[3. Subnetting — Dividing Networks into Smaller Worlds](#)

[The Core Idea Behind Subnetting](#)

[The Logic Behind Host Allocation](#)

[Subnetting with Python](#)

The internet feels invisible most of the time. You open a browser. You send a message. You watch a video. And somehow, everything just works.

But underneath all of that convenience is something extremely important: **Addresses**. Because computers cannot communicate randomly. They need identities. They need destinations. They need a way to know: ***“Where should this information go?”*** → That is exactly what an IP Address is.

An IP Address (Internet Protocol Address) is a unique identifier assigned to devices connected to a network.

Think of it like a digital home address. Without it, devices would have no idea where to send data.

And in Python, we can actually work with IP Addresses directly using the built-in `ipaddress` module. No extra installation. No external libraries. Just Python itself.

1. IPv4 Addresses

Most modern beginner networking examples use **IPv4**. An IPv4 Address is made from **32 bits**, usually displayed in decimal form like this:

192.168.1.10/24

At first glance, it just looks like numbers separated by dots. But underneath? It is actually binary. Every section is called an **octet**, and each octet contains 8 bits.

- So: 4 octets → 8 bits each → Total = 32 bits
- Which means: $2^{32} = 4,294,967,296$ possible IPv4 addresses can exist. That number is huge, but not infinite. Which is one reason IPv6 eventually became necessary.

Creating an IPv4 Address in Python

Python treats IP Addresses as actual objects.

```
# IPv4 Address with Python
from ipaddress import IPv4Address
ip = IPv4Address("192.168.12.10")
print(ip)
```

192.168.12.10

Simple, but important. Notice that we are not storing the IP as just a normal string. We are storing it as an `IPv4Address` object.

Which means Python now understands how to compare addresses, how to sort them, how to validate them, how to calculate networks from them. And that becomes extremely useful.

Comparing IPv4 Addresses

Since Python understands IP Addresses as structured objects, we can compare them directly.

```
# Compare two IPv4 Addresses
from ipaddress import IPv4Address
ip1 = IPv4Address("192.168.12.10")
ip2 = IPv4Address("220.168.13.134")
print(ip1 < ip2)
print(ip1 > ip2)
```

True

False

This works because IPv4 Addresses internally behave like integers. The address: `192.168.12.10` is ultimately converted into a 32-bit numeric value. So Python compares the numerical values behind the scenes.

Sorting IPv4 Addresses

Once addresses become objects, sorting becomes natural too.

```
# Return tuples of IPv4 Address objects
from ipaddress import IPv4Address
addresses = (IPv4Address("92.168.100.25"), IPv4Address("19
6.200.64.221"),
            IPv4Address("176.255.153.159"), IPv4Address("2
03.166.189.66"),
            IPv4Address("210.172.195.17"), IPv4Address("18
0.15.76.0"),
            IPv4Address("142.31.100.44"), IPv4Address("17
2.16.10.5"),
            IPv4Address("172.31.10.33"), IPv4Address("175.
64.10.17"))

for i in sorted(addresses):
    print(i)
```

```
92.168.100.25
142.31.100.44
172.16.10.5
172.31.10.33
175.64.10.17
176.255.153.159
180.15.76.0
196.200.64.221
203.166.189.66
210.172.195.17
```

And this introduces an important networking idea: IP Addresses are not random values. They exist inside structured ranges and hierarchies. That naturally leads us to networks.

2. Networks, CIDR, and Host Calculations

An individual IP Address identifies one device. But networks are about groups. A **network** is simply a collection of IP Addresses belonging to the same

address range.

For example: 192.4.2.0/24 → This does not represent one machine. It represents an entire network block.

Understanding CIDR Notation

The /24 part is called the **CIDR notation**. CIDR tells us: ***“How many bits belong to the network portion?”***

For IPv4:

- Total bits = 32
- Network bits = CIDR value
- Remaining bits = host bits

So for /24: $32 - 24 = 8$ → There are 8 host bits remaining. And those host bits determine how many devices the network can contain.

What makes CIDR interesting is that every increase in the prefix length reduces the available host space.

- A /25 network has fewer hosts than a /24 .
- A /26 network has fewer hosts than a /25 .
- And so on.

This happens because more bits are reserved for identifying the network, leaving fewer bits available for devices.

You can think of CIDR as a balancing act:

- Larger prefix → More networks, fewer hosts
- Smaller prefix → Fewer networks, more hosts

To better visualize this, let us generate the binary subnet mask associated with each CIDR value.

CIDR Notation and Binary Subnet Masks

```
for cidr in range(24, 33):
    mask = ("1" * cidr).ljust(32, "0")

    binary_mask = ".".join(mask[i:i+8]
                            for i in range(0, 32, 8))
    print(f"/{cidr} -> {binary_mask}")
```

```
/24 → 11111111.11111111.11111111.00000000
/25 → 11111111.11111111.11111111.10000000
/26 → 11111111.11111111.11111111.11000000
/27 → 11111111.11111111.11111111.11100000
/28 → 11111111.11111111.11111111.11110000
/29 → 11111111.11111111.11111111.11111000
/30 → 11111111.11111111.11111111.11111100
/31 → 11111111.11111111.11111111.11111110
/32 → 11111111.11111111.11111111.11111111
```

Notice how each step simply turns one additional host bit into a network bit. That tiny change dramatically affects the number of available addresses.

Working with Networks in Python

Now that we understand CIDR notation, we can let Python calculate the details for us automatically.

```
# Network of IP Addresses
from ipaddress import IPv4Network
network = IPv4Network("192.4.2.0/24")
print(network)
print("Address:", network.network_address)
print("CIDR:", network.prefixlen)
print("Number of addresses:", network.num_addresses)
```

```
192.4.2.0/24
Address: 192.4.2.0
CIDR: 24
Number of addresses: 256
```

Python is doing much more than displaying an address. It understands the network boundary, the prefix length, the total number of addresses, and the address range represented by that network.

And because every CIDR value corresponds to a different number of hosts, we can quickly compare them.

```
# Number of Addresses for Different CIDR Values
from ipaddress import IPv4Network

for cidr in range(24, 33):
    network = IPv4Network(f"192.168.1.0/{cidr}", strict=False)
    print(f"/{cidr}\t {network.num_addresses} addresses")
```

```
/24 256 addresses
/25 128 addresses
/26 64 addresses
/27 32 addresses
/28 16 addresses
/29 8 addresses
/30 4 addresses
/31 2 addresses
/32 1 addresses
```

The pattern becomes immediately visible. Every time the CIDR increases by one: $2^{(32-CIDR)}$ is cut in half.

This is one of the most important ideas in subnetting. A single bit may seem insignificant. But in networking, one bit can double—or halve—the size of an entire network.

IP Address Classes

Before CIDR became the dominant way of organizing networks, IPv4 addresses were commonly divided into predefined groups called **Classes**.

The idea was simple. Not every network needs the same number of devices. A small company might need only a few hundred addresses. A large organization might need millions.

So IPv4 originally categorized addresses into different classes based on their first octet.

Class	First Octet Range	Default CIDR	Purpose
A	1 – 126	/8	Very Large Networks
B	128 – 191	/16	Medium Networks
C	192 – 223	/24	Small Networks
D	224 – 239	N/A	Multicast
E	240 – 255	N/A	Experimental

For example:

- `10.0.0.1` belongs to Class A
- `172.16.0.1` belongs to Class B
- `192.168.1.1` belongs to Class C

Nowadays, CIDR is much more flexible and largely replaces the old class system. However, you will still encounter Class A, B, and C networks in networking books, certifications, and legacy systems.

Determining the Class of an IP Address

Python's `ipaddress` module does not directly provide an IP class.

But since classes depend only on the first octet, we can calculate them ourselves.

```
from ipaddress import IPv4Address

def get_ip_class(ip):
    first_octet = int(str(ip).split(".")[0])

    if 1 <= first_octet <= 126:
        return "Class A"
    elif 128 <= first_octet <= 191:
        return "Class B"
    elif 192 <= first_octet <= 223:
        return "Class C"
    elif 224 <= first_octet <= 239:
        return "Class D"
    else:
        return "Class E"

addresses = [IPv4Address("10.20.30.40"),
             IPv4Address("172.16.5.10"),
             IPv4Address("192.168.1.100"),
             IPv4Address("230.10.20.30")]

for ip in addresses:
    print(ip, "->", get_ip_class(ip))
```

10.20.30.40 → Class A

172.16.5.10 → Class B

192.168.1.100 → Class C

230.10.20.30 → Class D

Although classes are no longer the primary method of network design, understanding them helps explain the historical evolution of IPv4 addressing.

Public and Private IP Addresses

Not every IP Address on Earth can be used directly on the Internet. In fact, most devices around you right now are probably using **Private IP Addresses**. Your laptop. Your phone. Your smart television. Your printer.

All of them likely communicate through private addresses inside your local network.

- A **Public IP Address** is globally unique and routable on the Internet.
- A **Private IP Address** is reserved for internal networks and cannot be reached directly from the public Internet.

Three IPv4 ranges are reserved for private use:

Private Range	CIDR
10.0.0.0 – 10.255.255.255	/8
172.16.0.0 – 172.31.255.255	/12
192.168.0.0 – 192.168.255.255	/16

If you have ever seen addresses such as: `192.168.1.10` ; `172.16.5.20` ; `10.0.0.50` , you were looking at private addresses. And that is completely normal. Most home and office networks use them every day.

- **Identifying Public and Private Addresses:** One useful feature of Python's `ipaddress` module is that it can determine whether an address is private automatically.

```

from ipaddress import IPv4Address

addresses = [IPv4Address("192.168.1.10"),
             IPv4Address("10.20.30.40"),
             IPv4Address("172.20.10.5"),
             IPv4Address("8.8.8.8"),
             IPv4Address("1.1.1.1")]

for ip in addresses:
    print(f"{ip} -> Private" if ip.is_private else f"{ip} -> Public")

```

```

192.168.1.10 -> Private
10.20.30.40 -> Private
172.20.10.5 -> Private
8.8.8.8 -> Public
1.1.1.1 -> Public

```

This distinction is extremely important in networking. Private addresses allow billions of devices to exist inside local networks without consuming public IPv4 addresses.

Combined with technologies such as NAT (Network Address Translation), they became one of the key reasons IPv4 remained usable long after experts predicted it would run out.

And when analyzing a network, one of the first questions engineers often ask is: ***Is this address public, or private?*** Because that simple answer immediately reveals how the device communicates with the rest of the world.

Subnet Masks and Host Masks

CIDR notation also corresponds to something called a **Subnet Mask**. The subnet mask determines which bits belong to the network and which bits belong to hosts

For `/24`:

```
11111111.11111111.11111111.00000000
```

becomes:

255.255.255.0

The leading 1s represent network bits. The remaining 0s represent host bits.

Python can calculate this automatically.

```
# Network of IP Addresses - Subnet Mask
from ipaddress import IPv4Network
network = IPv4Network("192.4.2.0/24")
print("CIDR:", network.prefixlen)
print("Subnet Mask:", network.netmask)
print("Host Mask:", network.hostmask)
```

CIDR: 24

Subnet Mask: 255.255.255.0

Host Mask: 0.0.0.255

The **Host Mask** is essentially the inverse of the subnet mask. It highlights the bits reserved for devices inside the network.

Calculating Hosts

This is where networking becomes mathematical. The number of hosts depends entirely on the number of host bits.

The formula is essentially:

$$2^{(32-CIDR)}$$

But in most networks: 1 address is reserved for the network itself; 1 address is reserved for broadcasting. So usable hosts become:

$$2^{(32-CIDR)} - 2$$

Python can calculate all of this automatically.

```
# Network of IP Addresses - Usable Hosts
from ipaddress import IPv4Address, IPv4Network
network1 = IPv4Network("192.168.10.0/24")
print(network1)
print("Host Bits:", 32 - network1.prefixlen)
print("Total Hosts:", network1.num_addresses)
print("Usable Hosts:", network1.num_addresses - 2)
network2 = IPv4Network("172.16.0.0/20")
print(network2)
print("Host Bits:", 32 - network2.prefixlen)
print("Total Hosts:", network2.num_addresses)
print("Usable Hosts:", network2.num_addresses - 2)
```

```
192.168.10.0/24
Host Bits: 8
Total Hosts: 256
Usable Hosts: 254
172.16.0.0/20
Host Bits: 12
Total Hosts: 4096
Usable Hosts: 4094
```

Notice how dramatically the number changes. A small CIDR adjustment can completely transform the size of a network. That is why subnetting matters so much in real-world networking.

Returning Individual Host Addresses

Sometimes, we do not just want the network. We want every address inside it. Python allows iteration through all addresses in a network.

```
# Network of IP Addresses - Host Addresses
from ipaddress import IPv4Network

number_per_Line = 10
count = 0
network1 = IPv4Network("192.168.10.0/24")
for i in network1:
```

```
print(i, end="\t")
count += 1
# When the count reaches 10
if (count % number_per_Line) == 0:
    print()
```

This is especially useful for scanning ranges, automation scripts, subnet analysis, network simulations, cybersecurity tools

And suddenly, IP Addresses stop feeling like random numbers. They become structured systems. Logical systems. Mathematical systems.

3. Subnetting — Dividing Networks into Smaller Worlds

So far, we have treated a network as one large block of addresses. But real-world networks rarely stay that simple.

Because imagine this situation:

- One department needs 400 hosts
- Another only needs 20
- Another needs 6
- Another needs 100

If we place everything into one giant network, things become messy very quickly. Too many broadcasts. Too much wasted address space. Too little organization.

So networking engineers do something smarter: They divide one large network into multiple smaller networks. This process is called **Subnetting**.

The Core Idea Behind Subnetting

Subnetting is essentially the art of partitioning networks. We take one larger address block and divide it into smaller subnetworks that fit different requirements.

For example: A `/24` network contains $2^{(32-24)} = 256$ total addresses.

But maybe we do not need 256 hosts in one place. Maybe we need 60 hosts, 30 hosts, 14 hosts, 6 hosts

Instead of wasting huge portions of the network, we create smaller subnetworks that fit each requirement more efficiently. This approach is called **Variable Length Subnet Masking (VLSM)**. Different subnet sizes for different needs.

The Logic Behind Host Allocation

Before subnetting begins, we first determine: ***"How many addresses does each subnet actually require?"***

The subnet must always have enough space for usable hosts, network address, and broadcast address. Which means the allocated size is always a power of 2. Examples:

Required Hosts	Allocated Block
6	8
14	16
30	32
60	64

Notice something interesting. The allocation is always slightly larger than the requested hosts. Because networking structures rely heavily on binary boundaries.

Subnetting with Python

Python can automate subnet calculations surprisingly well.

```
# Network of IP Addresses - Subnetting
from ipaddress import IPv4Network
from math import log2
```

```
def get_required_hosts(hosts):
    # Minimum Hosts required: [2^2, 2^3, 2^4, ...]
```

```

    available_hosts = [4, 8, 16, 32, 64, 128, 256, 512, 1024]
    required_hosts = []
    # Get the required hosts
    for host in hosts.values():
        for available in available_hosts:
            # Break the loop if the available_hosts greater
            # than
            # the hosts is found
            if available > host:
                required_hosts.append(available)
                break
    return required_hosts

```

```

def get_cidr(required_hosts):
    return [32 - int(log2(hosts)) for hosts in required_hosts]

```

```

def subnetting(network, hosts):
    # Optional: Sort the subnets in descending order
    # Prioritize the one with the largest required hosts
    required_hosts = get_required_hosts(hosts)
    cidr = get_cidr(required_hosts)
    current_address = network.network_address
    for c, h in zip(cidr, hosts):
        # Set network address
        subnet = IPv4Network(f"{current_address}/{c}", strict=False)
        # get broadcast subnet
        network_addr = subnet.network_address
        broadcast_addr = subnet.network_address + (2 ** (32 - c) - 1)
        # usable Ips
        first_addr, last_addr = network_addr + 1, broadcast_addr - 1
        print(f"{h}: {first_addr} - {last_addr}")
        # Update the next address, 'last_addr + 2 to exclude

```



```
e the broadcast_addr
    # of the previous network, and the network_addr of
the current network
    current_address = last_addr + 2
```

```
print("+ Example 1")
network = IPv4Network("192.168.10.0/24")
hosts = {"Network A": 60, "Network B": 30,
        "Network C": 14, "Network D": 6}
subnetting(network, hosts)
```

+ Example 1

Network A: 192.168.10.1 - 192.168.10.62

Network B: 192.168.10.65 - 192.168.10.94

Network C: 192.168.10.97 - 192.168.10.110

Network D: 192.168.10.113 - 192.168.10.118

```
print("+ Example 2")
network = IPv4Network("172.16.0.0/20")
hosts = {"Network A": 400, "Network B": 200,
        "Network C": 100, "Network D": 50,
        "Network E": 25}
subnetting(network, hosts)
```

+ Example 2

Network A: 172.16.0.1 - 172.16.1.254

Network B: 172.16.2.1 - 172.16.2.254

Network C: 172.16.3.1 - 172.16.3.126

Network D: 172.16.3.129 - 172.16.3.190

Network E: 172.16.3.193 - 172.16.3.222

This code performs several important networking tasks automatically:

- Determines the minimum subnet size required
- Converts host requirements into CIDR values
- Calculates subnet boundaries
- Finds usable IP ranges

- Moves to the next valid subnet address

And underneath all of this, binary mathematics quietly controls everything.

CIDR Calculation

The CIDR value is calculated from the subnet size.

Formula: $CIDR = 32 - \log_2(Hosts)$

For example: If a subnet requires 64 addresses $\rightarrow 32 - \log_2(64) = 26$. So the subnet becomes `/26`.

Which means: 26 network bits, 6 host bits $\rightarrow$ And: $2^6 = 64$ addresses become available.

Subnetting is not just a classroom exercise. It is one of the core foundations of real networking. It allows engineers to organize departments, separate devices logically, improve security, reduce broadcast traffic, conserve address space, and design scalable infrastructures

Without subnetting, large networks become chaotic extremely quickly. And this is one of the fascinating things about networking: ***At the surface, the internet feels infinite. But underneath? Everything is carefully divided. Carefully calculated. Carefully structured through mathematics and binary logic.***

Summary

IP Addresses are one of the invisible foundations of modern computing. Every connection, every website, every online interaction depends on them.

And Python gives us surprisingly powerful tools to work with them directly.

Using the `ipaddress` module, we can:

- Create and validate IPv4 Address objects
- Compare and sort IP Addresses
- Distinguish between Public and Private IP Addresses
- Understand the historical IPv4 Class system (A, B, C, D, and E)
- Create and analyze IPv4 Networks

- Interpret CIDR notation and network prefixes
- Convert CIDR values into binary subnet masks
- Calculate the number of addresses available in a network
- Determine subnet masks and host masks
- Calculate usable host ranges
- Iterate through network addresses and hosts
- Perform subnetting using Variable Length Subnet Masking (VLSM)
- Allocate address blocks based on host requirements
- Design smaller subnetworks from larger network ranges

But beyond the code, there is a deeper idea here. Networking is not just about devices. It is about organization. Structure. Relationships between addresses.

And once you start understanding CIDR, subnet masks, and host calculations, you begin seeing the internet less like magic, and more like a carefully engineered system of mathematical communication.

References

<https://docs.python.org/3/howto/ipaddress.html>

<https://realpython.com/python-ipaddress-module/>

Next Section:

</> [27. Python: Drawing with Turtle Graphics](#)